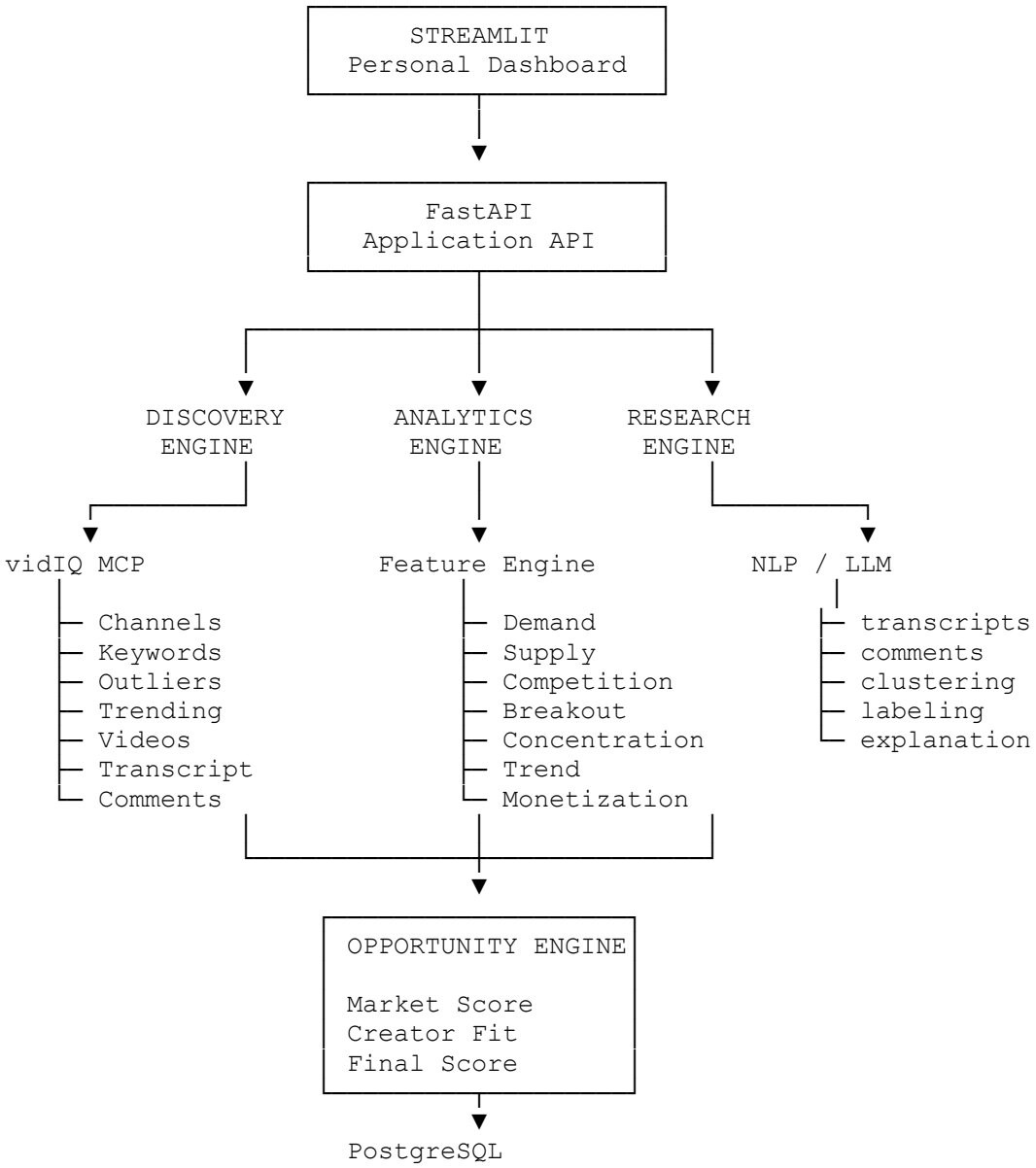
Arquitectura V1



La fuente principal de inteligencia sería **vidIQ**, porque actualmente expone mediante MCP datos de canales, keywords, outliers, tendencias, transcripts y análisis de videos, y está pensado explícitamente para funcionar con Claude y Codex.

---

1. Capa de presentación

Para la V1 usaría:

```
Streamlit
```

No React.

No Next.js.

No frontend separado.

Tu usuario eres tú mismo y lo que quieres validar es el motor.

La pantalla inicial podría ser:

```
NICHE RADAR
──────────────────────────────────────

Market:
[ English ▼ ]

Content:
[ Long-form ▼ ]

Channel type:
[ Faceless ▼ ]

Candidate niches:

[ Space mysteries        ]
[ Ancient history        ]
[ AI business stories    ]
[ Financial disasters    ]

[ ANALYZE ]
```

Resultado:

```
NICHE                  MARKET     FIT        FINAL
──────────────────────────────────────────────────
Space Mysteries          84        94          89
Ancient History          79        91          85
AI Business Stories      91        72          82
Financial Disasters      86        68          77
```

Después haces click en uno y ves el análisis profundo.

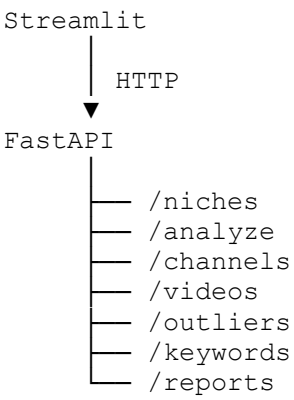
---

2. FastAPI como capa de aplicación

Aunque Streamlit podría llamar directamente Python, **yo sí pondría FastAPI desde la V1**.

Razón: quieres consolidar software engineering con Claude Code y Codex.

Arquitectura:



Ejemplo:

```
POST /api/v1/niches/analyze
```

Request:

```
{
  "niches": [
    "space mysteries",
    "ancient history"
  ],
  "language": "en",
  "content_type": "long_form",
  "faceless": true
}
```

Response:

```
{
  "results": [
    {
      "niche": "space mysteries",
      "market_score": 84.2,
      "creator_fit": 93.7,
      "final_score": 88.9,
      "recommendation": "ENTER"
    }
  ]
}
```

Esto te permite cambiar Streamlit por React en el futuro sin tocar el core.

---

3. Integration Layer

Aquí está una decisión arquitectónica importante.

Nunca dejaría que el resto de tu código llame directamente a vidIQ.

Crearía adapters.

```
integrations/
│
├── vidiq/
│   ├── client.py
│   ├── channels.py
│   ├── videos.py
│   ├── keywords.py
│   ├── outliers.py
│   └── comments.py
│
├── youtube/
│   └── client.py
│
└── trends/
    └── client.py
```

El dominio simplemente pide:

```
channel_provider.search(...)
```

No sabe si detrás está:

```
vidIQ
TubeLab
YouTube
otro proveedor
```

Eso te permitirá cambiar proveedores posteriormente.

Interfaces

Conceptualmente:

```
class ChannelProvider:
    def search_channels(...): ...

class VideoProvider:
    def search_videos(...): ...

class KeywordProvider:
    def research_keywords(...): ...
```

```
class OutlierProvider:
    def find_outliers(...): ...
```

Después:

```
VidIQChannelProvider
VidIQVideoProvider
VidIQKeywordProvider
```

Esta separación es muy importante.

---

4. Discovery Engine

Su misión no es puntuar todavía.

Su misión es:

convertir un nicho en un universo comparable de canales, videos, keywords y subnichos.

Pipeline:

```
"space mysteries"
        │
        ▼
keyword expansion
        │
        ▼
channel discovery
        │
        ▼
video discovery
        │
        ▼
outlier discovery
        │
        ▼
similar channels/videos
        │
        ▼
candidate universe
```

Aquí usarías principalmente:

```
vidIQ

channel search
keyword research
outliers
trending videos
```

```
similar videos
similar channels
```

vidIQ define sus outliers como videos que superan significativamente la performance habitual del propio canal y expone señales como Outlier Score, views, VPH, engagement y subscriber count.

Resultado de Discovery

Algo normalizado:

```
NicheUniverse
│
├── 120 channels
├── 1,500 videos
├── 250 keywords
├── 130 outliers
└── 14 candidate subniches
```

Ese sería el input del Analytics Engine.

---

5. PostgreSQL como única base de datos

No metería ahora:

```
MongoDB
Elasticsearch
Neo4j
Redis
vector database independiente
```

Demasiado.

Usaría:

```
PostgreSQL
+
pgvector
```

y listo.

Tablas iniciales

```
niches
channels
videos
```

```
keywords
outliers
comments
transcripts

niche_channels
niche_videos
video_topics

analysis_runs
feature_values
opportunity_scores
recommendations
```

Más concretamente:

```
channels
─────────────────────────
id
external_id
name
subscribers
avg_views
country
language
faceless
created_at
last_seen_at
videos
─────────────────────────
id
external_id
channel_id
title
published_at
views
likes
comments
duration
outlier_score
vph
analysis_runs
─────────────────────────
id
niche_id
started_at
completed_at
version
config_json
```

Esto último es importante.

Cada análisis debe ser reproducible.

---

6. Raw vs normalized data

No guardaría solamente:

```
views = 124000
```

Guardaría también el resultado original del proveedor.

Ejemplo:

```
raw_provider_payload
```

o una tabla:

```
raw_events
```

con JSONB.

Arquitectura:

```
EXTERNAL API
     │
     ▼
RAW RESPONSE
     │
     ▼
NORMALIZER
     │
     ▼
DOMAIN MODEL
```

Por ejemplo:

```
vidIQ
{
   weird_provider_field: ...
}
```

se transforma en:

```
Video(
    views=...,
    subscribers=...,
    outlier_score=...
)
```

Si mañana cambia la API, tu dominio sigue estable.

---

7. Feature Engine

Aquí empieza realmente tu Data Science.

Tendría un paquete independiente:

```
features/
│
├── demand.py
├── supply.py
├── breakout.py
├── competition.py
├── concentration.py
├── trend.py
├── evergreen.py
├── monetization.py
└── production.py
```

Cada módulo recibe datos y devuelve features.

Demand

Podría comenzar con:

```
median_views
p75_views
p90_views
median_vph
keyword_volume
keyword_momentum
```

Ejemplo:

```
DemandFeatures(
    median_views=84000,
    p90_views=640000,
    median_vph=117,
    keyword_score=78
)
```

---

8. Supply

Features:

```
videos_7d
```

```
videos_30d
videos_90d

active_channels_30d

videos_per_channel
```

Y:

SupplyGrowth=SupplyrecentSupplypreviousSupplyGrowth = \frac{Supply_{recent}}{Supply_{previous}}

---

9. Breakout

Esta posiblemente será una de las features principales.

```
outliers_30d

small_channel_outliers

median_outlier_score

P90_outlier_score
```

y:

BreakoutRate=SmallChannelOutliersVideosFromSmallChannelsBreakoutRate = \frac{SmallChannelOutliers}{VideosFromSmallChannels}

También:

ViewsSubscriberRatio=ViewsSubscribersViewsSubscriberRatio = \frac{Views}{Subscribers}

como señal secundaria.

---

10. Competition

No definiría competencia como:

```
number of videos
```

Eso sería demasiado pobre.

Crearía:

```
CompetitorCount
MedianSubscribers
MedianChannelAge
MedianViews
TopCreatorShare
SupplyGrowth
BreakoutAccessibility
```

Por ejemplo:

```
many competitors

pero

many small creators succeeding
```

puede seguir siendo atractivo.

---

11. Concentration

100% cálculo nuestro.

Tendrías:

```
HHI
Gini
Top3Share
Top5Share
Top10Share
```

Ejemplo:

```
Top5Share = 71%
```

→ incumbentes fuertes.

Mientras:

```
Top5Share = 19%
```

→ mercado fragmentado.

Esto puede darle a NicheRadar una métrica bastante diferenciadora.

---

12. Trend

Haría un composite de tres familias.

```
TrendScore
│
├── Keyword Trend
├── Video Trend
└── Creator Trend
```

Por ejemplo:

```
KeywordTrend
    rising keyword score

VideoTrend
    VPH
    number of trending videos

CreatorTrend
    channel growth
    breakout channel count
```

vidIQ ya proporciona keyword research y trending/outlier intelligence, por lo que puedes utilizarlo como fuente para varias de estas señales.

---

13. Subniche Engine

Esta parte sí la construiría desde V1 porque es uno de los componentes que hace interesante el proyecto.

Entrada:

```
titles
+
descriptions
+
transcripts
```

Después:

```
clean
  ↓
embeddings
  ↓
UMAP
  ↓
```

```
HDBSCAN
  ↓
cluster
  ↓
LLM label
```

Stack:

```
sentence-transformers
UMAP
HDBSCAN
```

Ejemplo:

```
SPACE
 │
 ├── Fermi Paradox
 ├── Alien Civilizations
 ├── Black Holes
 ├── Mars
 ├── Space Disasters
 └── Future Technology
```

Después puedes calcular Opportunity Score **por cluster**.

Ese debería ser uno de los principales diferenciadores del sistema.

---

14. pgvector

No pondría Pinecone ni Qdrant en la V1.

Tus embeddings caben perfectamente en PostgreSQL.

```
video_embedding VECTOR(...)
transcript_embedding VECTOR(...)
```

Después:

```
similar video
similar topic
similar subniche
```

Todo desde:

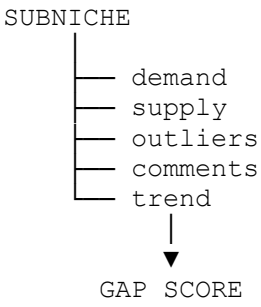
```
Postgres + pgvector
```

Menos infraestructura.

---

15. Content Gap Engine

Pipeline:



Una primera fórmula:

$$Gap = 0.30D + 0.25B + 0.20T + 0.15R + 0.10(100 - S)$$

donde:

```
D Demand
B Breakout
T Trend
R Audience Request
S Supply
```

Todavía heurístico.

Después puedes aprender los pesos.

---

16. Opportunity Engine

Aquí separaría desde el principio:

Market Score

```
Demand
Supply
Competition
Breakout
Concentration
Trend
```

```
Evergreen
Monetization
```

de:

Creator Fit

```
faceless suitability
production complexity
copyright risk
evergreen preference
budget
time/video
automation potential
personal interest
```

Entonces:

FinalScore=αMarketScore+(1−α)CreatorFitFinalScore = \alpha MarketScore + (1-\alpha)CreatorFit

Por ejemplo:

α=0.65\alpha = 0.65

inicialmente.

---

17. No usar ML para Opportunity Score todavía

Esto es importante.

V1

```
heuristics
+
normalization
+
weighted score
```

V2

Cuando tengas resultados:

```
historical feature vectors
        +
future performance
        ↓
supervised learning
```

Entonces podrías entrenar:

```
LightGBM
```

para predecir:

P(niche breakout next 30 days)P(\text{niche breakout next 30 days})

Pero hacerlo ahora sería **ML sin labels**.

No aportaría nada.

---

18. Recommendation Engine

Aquí sí puedes utilizar un LLM.

Pero no para decidir el score.

Primero:

```
DS
↓
84.6
```

Después el LLM recibe:

```
{
  "demand": 86,
  "breakout": 94,
  "competition": 61,
  "evergreen": 92,
  "creator_fit": 93
}
```

y genera:

```
Recommendation: ENTER

Why:

• Small faceless channels are repeatedly producing
```

```
  breakout videos.

• Demand is strong while content supply remains
  moderate.

• Videos show unusually strong evergreen behavior.

• Production is compatible with your faceless,
  long-form constraints.

Main risk:
Revenue potential appears moderate rather than high.
```

El LLM **explica**.

No inventa las métricas.

---

19. Production Feasibility Analyzer

No ejecutaría Video Watch sobre miles de videos.

Sería demasiado caro.

Pipeline inteligente:

```
1,500 videos
      ↓
metadata filters
      ↓
100 relevant
      ↓
outlier ranking
      ↓
20 representative
      ↓
Video Watch
```

Esto controla créditos.

vidIQ señala que Video Watch consume más créditos precisamente porque analiza el contenido del video, no solo metadata.

Resultado:

```
visual_complexity
editing_complexity
research_complexity
face_dependency
```

```
stock_dependency
animation_dependency
```

---

20. Job architecture

No haría que una petición HTTP ejecute:

```
search 300 channels
download 1,000 videos
cluster
analyze
LLM
```

sin separación.

Incluso para V1 utilizaría concepto de jobs.

```
POST /analyze
      |
      ▼
create AnalysisRun
      |
      ▼
pipeline executes
      |
      ▼
results stored
```

Puedes empezar síncronamente internamente y después pasar a workers.

V1

```
FastAPI
+
Python jobs
```

Cuando crezca

```
FastAPI
+
Redis
+
Celery / Dramatiq
```

Pero **no Redis en V1 todavía**.

---

21. Cache

Sí pondría cache desde el principio, pero en PostgreSQL.

Ejemplo:

```
provider_cache

provider
operation
request_hash
response_json
created_at
expires_at
```

Si preguntas dos veces:

```
space mysteries
```

no gastas dos veces credits.

Especialmente importante porque tu vidIQ conectado tiene una bolsa limitada de créditos compartidos para sus herramientas AI/MCP.

---

22. YouTube Data API: secundaria, no core

Sí obtendría una API key.

Pero inicialmente la utilizaría como fuente de apoyo para:

```
official metadata
video/channel IDs
search verification
```

No como fuente central del intelligence engine.

Desde junio de 2026, `search.list` tiene un bucket granular con 100 llamadas diarias por defecto y `videos.batchGetStats` dispone de un bucket predeterminado de 10.000 unidades/día.

Eso hace especialmente conveniente:

```
vidIQ
↓
discovery
```

```
YouTube API
↓
verification/enrichment
```

---

23. Google Trends lo pondría detrás de una interfaz

Algo como:

```
class TrendProvider:
    def interest_over_time(...): ...
    def related_queries(...): ...
```

Después:

```
DataForSEOTrendProvider
```

o futuro:

```
GoogleTrendsProvider
```

Así no atas la arquitectura a un proveedor.

---

24. Package architecture

Yo organizaría el repo así:

```
nicheradar/
│
├── apps/
│   ├── api/
│   │   └── main.py
│   │
│   └── dashboard/
│       └── app.py
│
├── src/
│   └── nicheradar/
│       │
│       ├── domain/
│       │   ├── models/
│       │   ├── entities/
│       │   └── enums/
│       │
│       ├── integrations/
```

```
│       │   ├── vidiq/
│       │   ├── youtube/
│       │   └── trends/
│       │
│       ├── discovery/
│       │   ├── channels.py
│       │   ├── videos.py
│       │   ├── keywords.py
│       │   └── outliers.py
│       │
│       ├── features/
│       │   ├── demand.py
│       │   ├── supply.py
│       │   ├── breakout.py
│       │   ├── competition.py
│       │   ├── concentration.py
│       │   ├── trend.py
│       │   ├── evergreen.py
│       │   └── monetization.py
│       │
│       ├── topics/
│       │   ├── embeddings.py
│       │   ├── clustering.py
│       │   └── labeling.py
│       │
│       ├── scoring/
│       │   ├── market.py
│       │   ├── creator_fit.py
│       │   └── opportunity.py
│       │
│       ├── recommendations/
│       │   └── engine.py
│       │
│       ├── repositories/
│       │
│       └── services/
│
├── tests/
│   ├── unit/
│   ├── integration/
│   └── e2e/
│
├── migrations/
│
├── notebooks/
│
├── docs/
│
├── docker-compose.yml
├── Dockerfile
├── pyproject.toml
└── README.md
```

---

25. Dependencias de V1

Mantendría el stack aproximadamente así:

```
Python 3.11+

uv

FastAPI
Uvicorn

Pydantic v2
pydantic-settings

SQLAlchemy 2
Alembic

PostgreSQL
pgvector

Polars
NumPy
SciPy

scikit-learn
sentence-transformers
UMAP
HDBSCAN

httpx

Streamlit

pytest
pytest-asyncio
mypy
ruff
pre-commit

Docker
GitHub Actions
```

No pondría inicialmente:

```
Kubernetes
Kafka
Airflow
Spark
feature store
Kubeflow
MLflow
vector SaaS
microservices
```

Todo eso sería arquitectura antes de tener problema.

---

26. Cómo fluiría una consulta real

Supón:

```
Analyze:

Space Mysteries
```

Paso 1

```
AnalysisRun created
```

Paso 2

Keyword expansion:

```
space mysteries
fermi paradox
alien civilizations
great filter
dark forest
...
```

Paso 3

Channel discovery:

```
faceless
long-form
English
active
```

Paso 4

Video discovery.

Paso 5

Outlier discovery.

Paso 6

Normalización y almacenamiento.

Paso 7

Feature calculations:

```
demand = 84
supply = 52
competition = 48
breakout = 93
concentration = 81
trend = 76
monetization = 62
```

Paso 8

Clustering:

```
Space Mysteries
│
├── Fermi Paradox
├── Great Filter
├── Dark Forest
├── Ancient Aliens
└── Simulation Theory
```

Paso 9

Score cada subnicho.

```
Great Filter        91
Dark Forest         87
Fermi Paradox       84
Simulation Theory   76
Ancient Aliens      61
```

Paso 10

Video Watch solo de candidatos top.

Paso 11

Creator Fit.

Paso 12

Final report.

---

27. Resultado final del sistema

No quiero que la V1 termine en un dashboard de charts.

Quiero que termine aquí:

```
═══════════════════════════════════════════
NICHE RADAR REPORT
═══════════════════════════════════════════

Market:
Existential Space Mysteries

FINAL SCORE                          88 / 100

Market opportunity                   84
Creator fit                          94

Recommendation:

                  ENTER

──────────────────────────────────────────

Best subniche

The Great Filter                     91

──────────────────────────────────────────

WHY

Demand                               HIGH
Competition                          MEDIUM
Breakout accessibility               VERY HIGH
Trend                                HIGH
Evergreen                            VERY HIGH
Monetization                         MEDIUM
Faceless suitability                 VERY HIGH
Production complexity                MEDIUM-LOW
Copyright risk                       LOW

──────────────────────────────────────────

EVIDENCE

Small-channel breakouts               23
Median small-channel outlier        6.8x
Median channel subscribers           27K
Median breakout video views         312K

──────────────────────────────────────────

SUGGESTED FORMAT

Video Essay / Documentary
18–26 minutes
Narrated faceless
1 upload/week
```

```
TOP CONTENT GAPS

1. The Great Filter
2. Dark Forest Theory
3. Civilizations before humanity
4. Self-replicating probes
5. Why civilizations disappear
```

Eso es el producto.

Qué considero realmente la V1

El límite lo pondría aquí:

```
                    V1

vidIQ
  +
Google Trends
  +
PostgreSQL
  +
Feature Engineering
  +
Topic Clustering
  +
Opportunity Score
  +
Streamlit
```

No generaría videos todavía.

No haría thumbnails.

No publicaría.

No construiría agentes autónomos complejos.

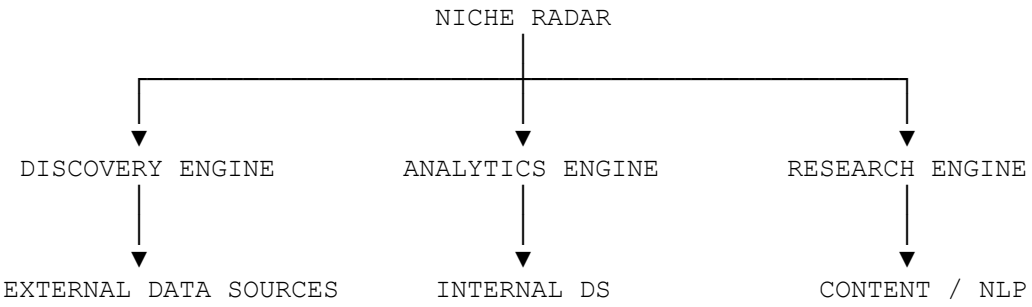
No entrenaría ML predictivo.

Primero quiero demostrar una sola cosa:

Que NicheRadar puede ordenar nichos y subnichos de una manera suficientemente buena como para cambiar tu decisión sobre qué canal crear.

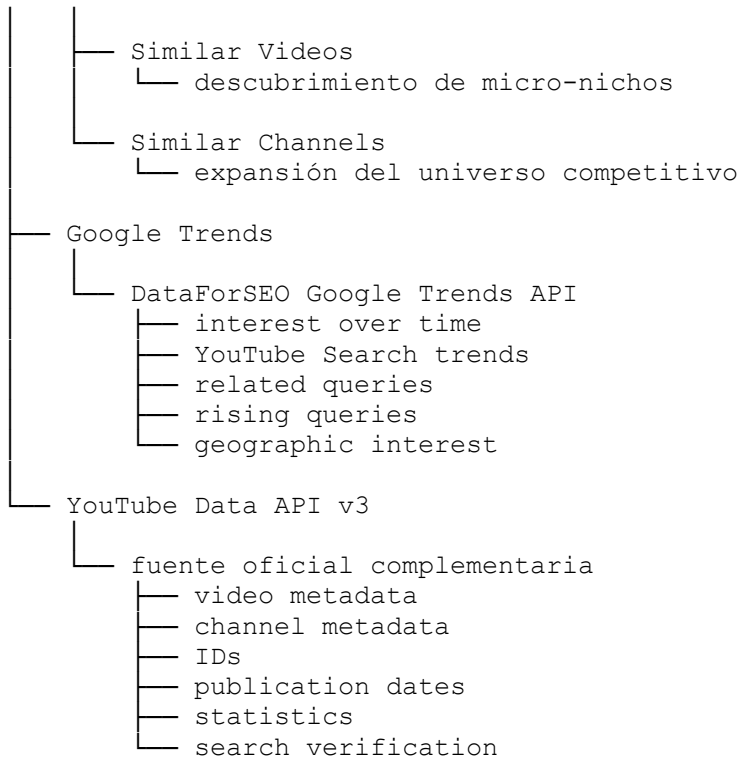
Una vez comprobado eso, la **V2 natural sería el backtesting**: guardar cada predicción y comprobar 30/60/90 días después si los nichos que marcamos como `ENTER` realmente superaron a los `WATCH` y `AVOID`. Ahí es donde este proyecto empezaría a transformarse de una herramienta de analytics en un auténtico sistema de Data Science predictivo.

Fuentes externas y herramientas por motor

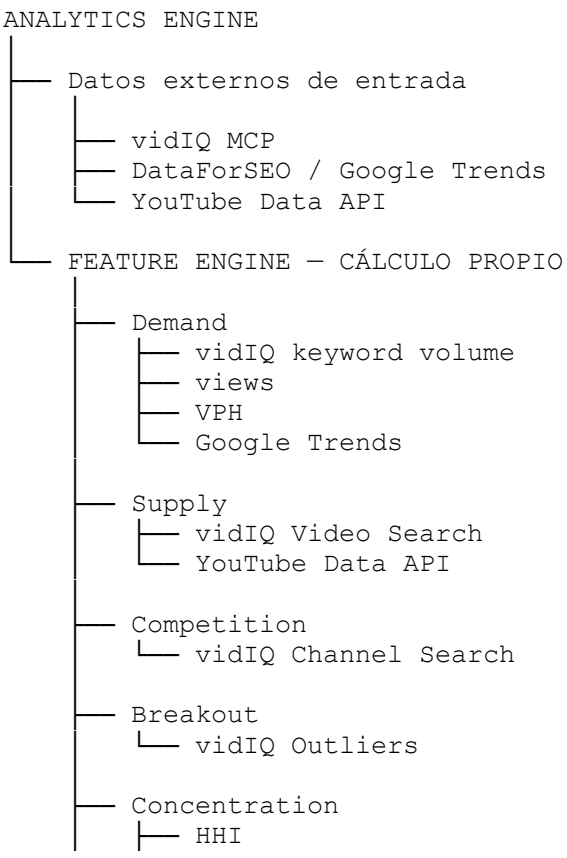


1. DISCOVERY ENGINE

```
DISCOVERY ENGINE
│
├── vidIQ MCP
│   │
│   ├── Channel Search
│   │   ├── canales por nicho
│   │   ├── subscribers
│   │   ├── average views
│   │   ├── crecimiento
│   │   ├── faceless
│   │   └── breakout channels
│   │
│   ├── Keyword Research
│   │   ├── keyword volume
│   │   ├── competition
│   │   ├── related keywords
│   │   ├── rising keywords
│   │   └── country-level signals
│   │
│   ├── Outliers
│   │   ├── outlier score
│   │   ├── views
│   │   ├── VPH
│   │   └── engagement
│   │
│   ├── Trending Videos
│   │   ├── videos actualmente acelerando
│   │   └── Views Per Hour
│   │
│   ├── Video Search
│   │   ├── videos por query
│   │   ├── fechas
│   │   ├── duración
│   │   └── metadata
```



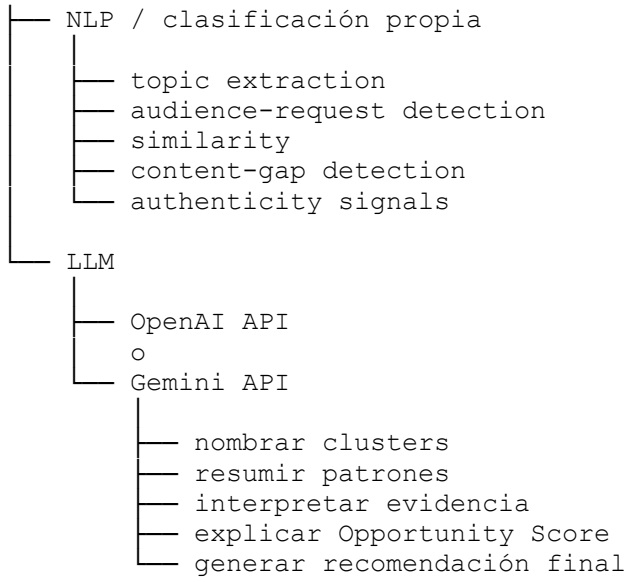
2. ANALYTICS ENGINE



```
│   ├── Gini
│   ├── Top-3 share
│   ├── Top-5 share
│   └── Top-10 share
│
├── Trend
│   ├── vidIQ Rising Keywords
│   ├── vidIQ Trending Videos
│   ├── VPH
│   └── Google Trends
│
├── Evergreen
│   ├── vidIQ Video Stats
│   └── vidIQ Channel Performance Trends
│
└── Monetization
    ├── vidIQ Earnings Estimate
    ├── vidIQ Channel Earnings
    └── opcional: Google Ads API
        ├── CPC
        └── advertiser competition
```

3. RESEARCH ENGINE

```
RESEARCH ENGINE
│
├── vidIQ MCP
│   │
│   ├── Video Transcript
│   │   └── texto completo del video
│   │
│   ├── Video Comments
│   │   └── comentarios y señales de audiencia
│   │
│   ├── Video Watch
│   │   └── análisis visual/estructural del video
│   │
│   ├── Similar Videos
│   │   └── contenido comparable
│   │
│   └── Similar Thumbnails
│       └── patrones visuales del nicho
│
├── Embeddings
│   │
│   └── sentence-transformers
│       ├── titles
│       ├── transcripts
│       ├── comments
│       └── video topics
│
├── Clustering
│   │
│   ├── UMAP
│   └── HDBSCAN
│
```



4. RESUMEN DE SERVICIOS DE TERCEROS

```
TERCEROS / APIs / MCP

1. vidIQ MCP
   → fuente principal de inteligencia YouTube

2. DataForSEO Google Trends API
   → tendencias y demanda externa

3. YouTube Data API v3
   → metadata oficial y verificación

4. OpenAI API o Gemini API
   → NLP avanzado, labeling y explicación

5. Google Ads API                          [opcional V1]
   → CPC y valor comercial de keywords
```

5. QUÉ NO ES SERVICIO EXTERNO

```
LO CALCULAMOS NOSOTROS

Demand Score
Supply Score
Competition Score
Breakout Rate
Views/Subscribers Ratio
HHI
Gini
Top-N Share
Trend Score
Evergreen Score
Monetization Score
Production Feasibility
```

Copyright Risk
Authenticity Risk
Content Gap Score
Market Score
Creator Fit
Opportunity Score
ENTER / WATCH / AVOID